

POLYTECHNIQUE
MONTRÉAL

INF 2010

Structures de données et algorithmes

Remise TP 4

Soumis par :
Kanzari, Yassmin, 2457562
Carpentier, Alex, 2447572

Dimanche, le 8 novembre 2026

Partie 2: Compression de Huffman

Après la décompression du fichier secret avec l'algorithme HuffmanCompression, on remarque que ce dernier a une taille de 5119982 octets. La commande file nous indique que c'est une image bmp de 2262x2261 pixels. Ceci peut être conclu car les deux premiers octets sont BM qui représente la signature d'un fichier BMP. Le fichier décompressé est une image de fleurs et de feuilles dans un pot à couvercle en noir et blanc. En effet, cette image et faites de 256 différents gris. Selon le tableau des fréquences, on remarque que l'octet 0 apparaît 3589078 fois. L'octet 0 représente la présence de la couleur noire.

Tableau 1 : La fréquence des octets dans le fichier secret.

Octet	Fréquence	Octet	Fréquence	Octet	Fréquence	Octet	Fréquence
0	3589078	64	4182	128	8766	192	10128
1	186	65	4205	129	9021	193	9624
2	173	66	4007	130	9486	194	9402
3	174	67	3 885	131	9425	195	9420
4	153	68	3766	132	9746	196	9030
5	170	69	3831	133	10114	197	8811
6	183	70	3661	134	10097	198	8696
7	213	71	3753	135	10054	199	8576
8	300	72	3731	136	10121	200	8296
9	295	73	3760	137	10388	201	8369
10	300	74	3789	138	10628	202	8019
11	381	75	3753	139	10521	203	7482
12	340	76	3790	140	10910	204	7354
13	355	77	3806	141	11340	205	7297
14	396	78	3776	142	11497	206	7313
15	417	79	3539	143	11850	207	7374
16	374	80	3410	144	11447	208	7513
17	401	81	3494	145	11267	209	7530
18	429	82	3532	146	11216	210	7620
19	414	83	3514	147	11217	211	7708
20	392	84	3528	148	12279	212	7987
21	401	85	3510	149	13037	213	8059
22	419	86	3650	150	13158	214	7843
23	435	87	3520	151	13053	215	7727
24	386	88	3531	152	13096	216	7246
25	451	89	3544	153	12654	217	7385
26	469	90	3688	154	12464	218	7307
27	490	91	3595	155	12604	219	7173
28	469	92	3715	156	12540	220	7026
29	452	93	3736	157	12641	221	7050
30	474	94	3837	158	12620	222	6961
31	447	95	3810	159	12353	223	7404
32	473	96	3876	160	12491	224	7400

33	516	97	4159	161	12684	225	7281
34	499	98	4421	162	12633	226	7473
35	625	99	4631	163	12672	227	7559
36	657	100	4704	164	12566	228	7721
37	607	101	4788	165	12632	229	7603
38	650	102	4905	166	12705	230	7418
39	703	103	5247	167	12980	231	7362
40	689	104	5399	168	13141	232	7340
41	666	105	5154	169	13819	233	6710
42	671	106	5330	170	13980	234	5913
43	792	107	5621	171	13227	235	6171
44	739	108	5672	172	12934	236	6060
45	743	109	6198	173	13230	237	5094
46	787	110	6329	174	13295	238	4278
47	908	111	6644	175	13294	239	4357
48	953	112	6494	176	14051	240	4487
49	1429	113	6559	177	13304	241	4748
50	1461	114	6624	178	13242	242	3308
51	1495	115	6866	179	13107	243	3179
52	1612	116	7355	180	13016	244	10800
53	1631	117	7587	181	12659	245	3
54	1754	118	7822	182	12543	246	3
55	1837	119	7925	183	12176	247	3
56	1864	120	7947	184	11764	248	3
57	2029	121	7930	185	11443	249	3
58	2153	122	8042	186	11230	250	4
59	2586	123	8359	187	11184	251	3
60	3299	124	8591	188	11115	252	3
61	3410	125	8510	189	10581	253	3
62	3604	126	8454	190	10559	254	4
63	3607	127	8486	191	10145	255	3

Partie 3 : Analyse de l'algorithme de Huffman

On a utilisé trois types de fichiers pour tester la performance de la compression tout en variant leurs tailles. On a donc opté pour 4 fichiers .txt. Le plus petit avait une taille de 1873 octets, le deuxième fait 847319 octets et le plus volumineux 6431207 octets. Ils contiennent tous des phrases aléatoires en français. On a aussi ajouté un fichier avec des caractères ascii de taille 523117 octets. Pour les fichiers binaires, ils sont aussi 4 fichiers. Le premier fichier binaire du tableau si dessous est un fichier contenant une suite des octets 0x00 et 0xFF tirés au hasard. Après les 3 autres fichiers sont formés de plusieurs octets aléatoires mais en quantité différente et ce pour avoir des tailles différentes. On a aussi utilisé le fichier secret donné pour la deuxième partie du tp.

Tableau 2: L'effet de la compression de fichiers de tailles différentes

Fichier	k	Original	Profondeur	Arbre	Données	Compressé	Ratio
texte_1873o	26	1873	10	33	941	977	52,16%
texte_847319o	44	847319	16	55	427420	427479	50,45%
texte_6431207o	44	6431207	16	55	3244431	3244490	50,45%
ascii_uniforme_523117o	95	523117	7	119	434656	434779	83,11%
bin_2octets_912481o	2	912481	1	3	114061	114067	12,50%
bin_aleatoire_1337o	254	1337	11	318	1318	1640	122,66%
bin_aleatoire_683941o	256	683941	8	320	683941	684265	100,05%
bin_aleatoire_7219853o	256	7219853	8	320	7219853	7220177	100,00%
fichier_secret	256	5 119 982	19	320	2 086567	2086891	40,76%

En calculant le ratio de compression des fichiers, il est possible de remarquer que les texte compresse plus que les fichiers binaires (50% contre 100%). En effet, il même possible d'affirmer que les fichiers binaires ne compressent presque jamais. Ceci peut être expliquer par les tendances des fichiers binaires. Dans notre cas, l'alphabet français ne contient que 26 lettres qui se répètent plus qu'une fois ainsi que des espaces et des ponctuations. Les binaires aléatoires ne compressent pas car leurs 256 valeurs sont équiprobables et tous les codes sont écrits sur 8 bits. Cependant, on remarque que l'image secrète du tp est compressible. En effet, le ratio de sa compression est de 40,8 % car l'octet 0 représente 70% des données comme le montre le tableau des fréquences. Il est possible aussi de remarquer que le ratio ne change pas selon la taille si on regarde les fichiers .txt. On peut même dire que le contraire est vrai et qu'un petit fichier peut grossir. Ceci est le cas du fichier aléatoire de 1337 octets qui est passé à 1640 octets après la compression. Pour un nombre de caractères différents (k) faible, la compression se fait très bien comme pour le binaire de 2 octets différents. Un nombre élevé n'est pas défavorable. Ce qui compte surtout, c'est la fréquence à laquelle chaque valeur apparaît. Ceci est démontré par l'image qui a 256 octets différents et un ratio de compression de 40%. L'arbre occupe [(10k -1)/8] octets. Il ne dépend que de k. Pour les gros fichiers, la taille de l'arbre est négligeable et les données représentent plus de 99,9 % du fichier compressé. Or que pour les petits fichiers, l'arbre occupe une plus grande place comme pour le binaire sur 1337 octets aléatoires, l'arbre fait 318 octets. Pour ce qui est de la profondeur, elle vaut 1 pour 2 symboles, 8 pour 256 symboles équiprobables, 16 pour le texte et 19 pour l'image. L'image a la plus grande profondeur car les octets très rares reçoivent des codes longs, alors que l'octet 0 en reçoit un d'un seul bit. Une grande profondeur veut donc dire qu'il y a de fortes inégalités. On peut donc confirmer que le taux de compression dépend de l'inégalité des fréquences et non du type et de la taille.

Hypothèse 1 : la construction de l'arbre est en O(k log k) avec la file et en O(k^2) sans la file. L'écart ne doit donc apparaître que si k est grand.

Hypothèse 2 : comme k ≤ 256 ne dépend pas de n, cette section est négligeable devant le comptage et l'encodage (O(n)). L'écart sur le temps total ne doit être visible que pour de petits fichiers à grand k.

Tableau 3 : Les temps de construction des fichiers avec ou sans fils

Fichier	n	k	Arbre avec file (µs)	Arbre sans file (µs)	Total avec file (ms)	Total sans file (ms)
texte_1873o	1873	26	3,2	2,7	0,01	0,02
bin_aleatoire_1337o	1337	254	41,6	170,6	0,06	0,19
ascii_uniforme_523117o	523117	95	33,7	58,4	5,24	4,13
bin_aleatoire_683941o	683941	256	45,6	154,1	8,14	7,85
texte_847319o	847319	44	9,6	10,8	4,20	4,25
bin_2octets_912481o	912481	2	3,6	16,9	3,85	4,01
image_secrete	5119982	256	44,0	125,3	28,79	28,55
texte_6431207o	6431207	44	22,8	24,5	36,24	33,93
bin_aleatoire_7219853o	7219853	256	58,0	173,6	81,68	80,90

Pour un grand k, la construction sans file est plus lente (174 µs contre 58 µs). Pour k = 95, l'écart est plus faible (58 µs contre 34 µs) et pour un k plus faible, les deux versions sont presque égales. L'écart grandit donc avec k. Ceci confirme la première hypothèse. De plus, pour le fichier de 1337 octets aléatoires, l'arbre représente environ 70 % du temps total et la version sans file est environ 3 fois plus lente (0,19 ms contre 0,06 ms). Dès 500 Ko, l'encodage prend le plus de temps. Les écarts entre els temps totaux sont donc trop petits ce qui confirme la deuxième hypothèse.

Avec des octets, k est plafonné à 256. La construction de l'arbre coûte au plus quelques centaines de µs, alors que l'encodage de n octets coûte des dizaines de ms. Le temps total est alors dominé par les étapes en O(n). L'écart n'est visible que pour les petits fichiers à grand k.